rsna-intracranial-aneurysm-detection

9th place solution

Rank	9
Team	Vibes and Genius Trade-Off
Author	Tom
Topic ID	611908
Version	Original

Acknowledgement

First, we thank the competition hosts @evancalabrese, @shosys and every person involved from RSNA in data preparation and competition related process, and Kaggle staff for organizing this competition.

Below, we introduce the solution of team **Vibes and Genius Trade-Off** -- @tom99763, @sersasj, @iamparadox, @chihantsai, @atom1231!

TL;DR

Our solution combines three complementary approaches:

- YOLO 2.5D with different backbones. Derived from BYU competition
- 3D CenterNet with 2D Effv2s extractor.
- Meta-classifiers (LightGBM, XGBoost, CatBoost)

The final probabilities are obtained by averaging the outputs of the YOLO 2.5D models, 3D CenterNet with 2D Effv2s extractor and the three meta-classifiers.

Here is the diagram of our approach.

We used two different variations of YOLO:

1. **YOLOv11m** - The standard YOLO11 medium model from Ultralytcs
2. **Custom YOLO with timm backbone** - YOLO architecture with `timm/tf_efficientnetv2_s.in21k_ft_in1k` as the backbone

For details about the YOLO+timm customization, please refer to the BYU writeup.

We treated each of the **13 vessel locations as separate bounding box classes**:

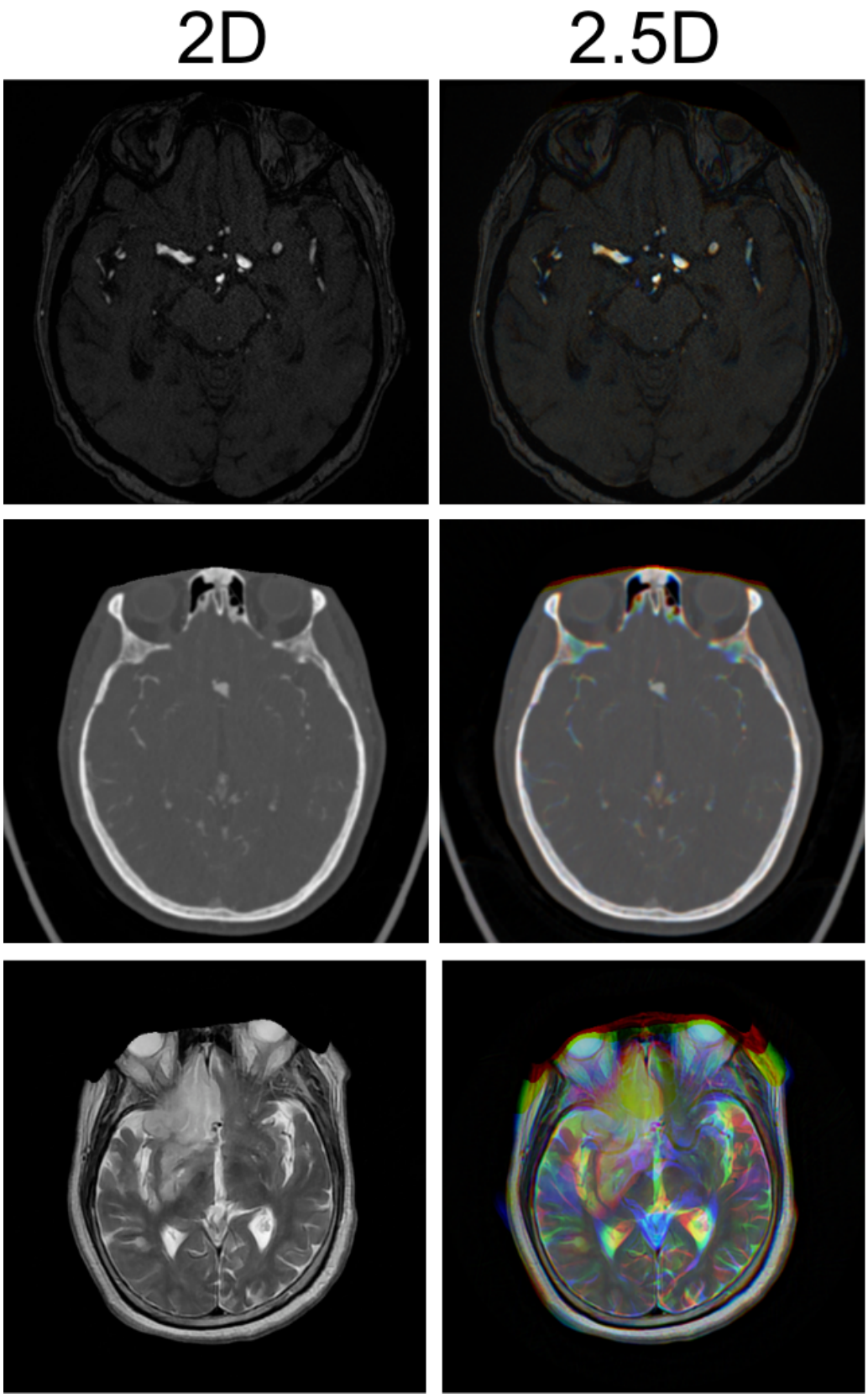
- Left/Right Infraclinoid Internal Carotid Artery
- Left/Right Supraclinoid Internal Carotid Artery
- Left/Right Middle Cerebral Artery
- Anterior Communicating Artery
- Left/Right Anterior Cerebral Artery
- Left/Right Posterior Communicating Artery
- Basilar Tip
- Other Posterior Circulation

2.5D Strategy

Images were resized to 512×512, normalized with min-max and converted to 2.5D variation:

```
R (Red channel) = slice i-1 G (Green channel) = slice i B (Blue channel) = slice i+1
```

The figure below shows examples of slices in 2D vs 2.5D variation:



As can be seen in the examples, the images vary significantly because **we didn't standardize Z spacing**.

I (@sersasj) spent 1-2 weeks experimenting with Z-axis resize, but results were consistently worse. Perhaps I was doing something wrong, but didn't have time to investigate further.

We were very reluctant to try 2.5D without proper Z-spacing resize (in my mind didn't seem a good idea - @sersasj). Nevertheless, the 2.5D approach even without Z-resampling improved results by 0.02+ in CV.

Training Configuration

Data Sampling Strategy:

For negative samples we took 10 evenly sampled slices per series.

Example: For a 100-slice series → slices [1, 11, 22, 33, 44, 56, 67, 78, 89, 99]

For positive samples we used all slices containing annotations

YOLO with timm/tf_efficientnetv2_s Backbone

YOLOv11m

We also modified the fitness function to include auc metric and prioritize mAP@50.

```
fitness =  × mAP@ +  × mAP@- +  × mAUC
```

Did this help? Maybe a little. In the BYU competition, we had already found that prioritizing mAP@50 over the Ultralytics standard 1.0×mAP@50-95 gave better results.

Adding AUC seemed like a good idea at the time since it's the competition metric, but we didn't investigate thoroughly whether it significantly improved performance. It's possible the benefit was marginal, but we kept it for consistency.

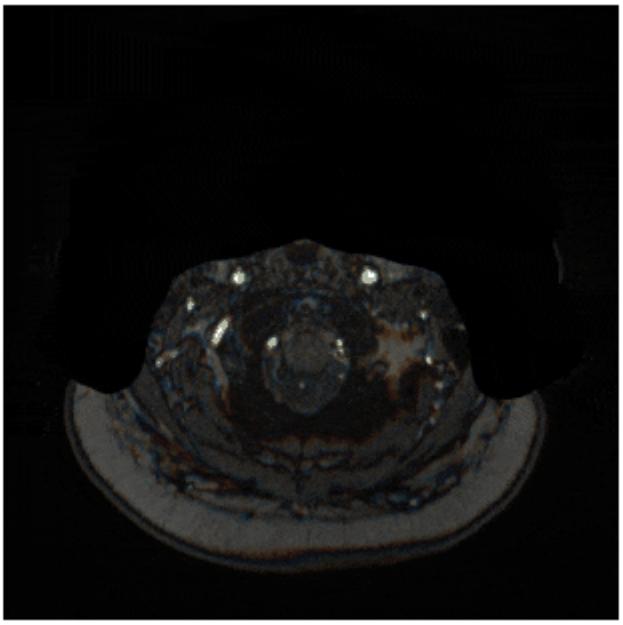
Training Hardware and Time

Hardware Setup	GPU	CPU	RAM	Time per Fold	Total (5 folds)
@sersasj	RTX 3090	Intel Core i5-12400F (12) @ 4.4GHz	32GB	4-5 hours	~24 hours

Hardware Setup	GPU	CPU	RAM	Time per Fold	Total (5 folds)
@iamparadox	RTX 4090	AMD Ryzen 9 7950X (32) @ 5.883GHz	64GB	~2 hours	~10 hours

Inference

For inference, we sort DICOM slices by spatial position (SliceLocation → ImagePositionPatient → InstanceNumber), create 2.5D triplets for slices 1 to N-1
Run batch inference across both models, extract max confidence per class and use max(localization_confidences) as overall aneurysm presence probability
The process can be observed in the gif below:



Cross-Validation Scores

folds were stratified using `MultilabelStratifiedKFold` to ensure balanced distribution across:

- Aneurysm presence
- All 13 vessel locations
- Modality

YOLO11m 2.5D Results

Fold	loc_macro_auc	cls_auc	combined_mean
fold0	0.8184	0.7549	0.7867
fold1	0.8114	0.7897	0.8005
fold2	0.8134	0.7827	0.7981
fold3	0.8162	0.7872	0.8017
fold4	0.8540	0.8281	0.8410

Fold	loc_macro_auc	cls_auc	combined_mean
Average	**0.8227**	**0.7885**	**0.8056**

EfficientNetV2-S 2.5D Results

Fold	loc_macro_auc	cls_auc	combined_mean
fold0	0.7978	0.7790	0.7884
fold1	0.8159	0.8269	0.8214
fold2	0.8155	0.7921	0.8038
fold3	0.8153	0.7907	0.8030
fold4	0.8499	0.8560	0.8529
Average	**0.8189**	**0.8090**	**0.8139**

Ensemble Results

Fold	loc_macro_auc	cls_auc	combined_mean
fold0	0.8393	0.7849	0.8121
fold1	0.8330	0.8291	0.8310
fold2	0.8399	0.8135	0.8267
fold3	0.8370	0.8093	0.8232
fold4	0.8738	0.8620	0.8679
Average	**0.8446**	**0.8198**	**0.8322**

What did not work:

In the initial stages of YOLO development @sersasj created an ensemble with 3xYolo11m and 0.69LB EfficientNetB2 public notebook. That gave us a score of 0.78 LB and put us in the top 3 in early stages of competition

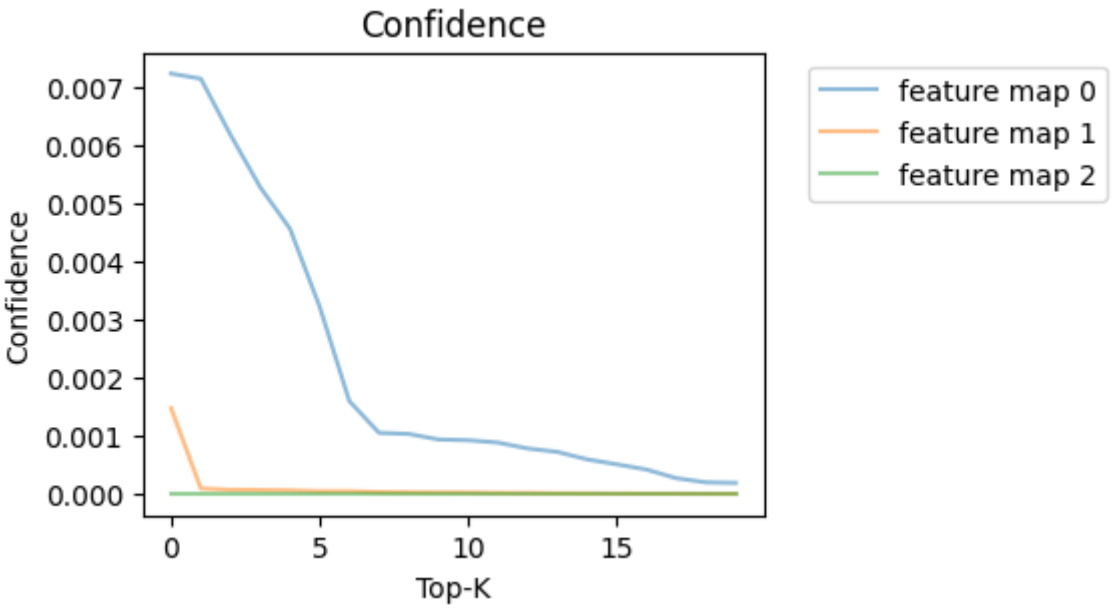
After a bit of probing the leaderboard, @iamparadox found out that YOLO's aneurysm classification AUROC was very low (around ~0.58). Then we started to develop models that would complement YOLO and boost the aneurysm classification AUROC. (We later discovered that the poor results of early YOLO development were due to an insufficient amount of negative samples in training.)

One of the class of models that we used was 2.5D models that just detects the presence of aneurysm in the given series with auxiliary segmentation head. These models were inspired by https://www.kaggle.com/code/hengck23/3d-unet-using-2d-image-encoder. These models in combination with yolo11m gave us a LB of 0.80.

These models then became obsolete when we merged with @chihantsai, @atom1231.

In the BYU competition, our best-performing backbone was `timm/convnextv2_base.fcmae_ft_in22k_in1k`. However, in the current competition, training with half precision caused the loss to quickly become NaN. Based on our investigation, this appears to be a common issue with ConvNeXt architectures. We hypothesize that if trained in full precision, this backbone could achieve performance comparable to YOLO11m and `timm/tf_efficientnetv2_s.in21k_ft_in1k`. Unfortunately, we were unable to verify this due to time constraints.

Modifying the neck and head was also considered. From what we investigate with @tatamikenn awesome reverse engineer notebook P3 features appeared to be the most utilized. We conducted a quick test on a single fold, but results on validation were more or less the same so we decided to move on.



We also try to use our 3d segmentation result to filter out invalid location prediction from yolo, but recall drops a lot.
Moreover, we tried extracting patches using YOLO and applying wavelet and log-polar transforms to the patches before predicting their classes; however, this approach did not yield good results.

EfficientV2s + 3D-CenterNet (Flayer)

This is a highly **experimental project**, made with a lot of help from AI. We started with the popular "LB 0.69 Public Notebook" , reconstructed the entire **training pipeline and data preprocessing** from it, and then incorporated a **CenterNet-like mechanism to detect the center of the aneurysms**.

Model architecture

We adopt a **slice-wise 2D encoder + shallow 3D head** design:

- **Encoder.** A 2D ImageNet-pretrained **EfficientNetV2-S** (`tf_efficientnetv2_s.in21k_ft_in1k`) from `timm` in features_only mode with the **penultimate** feature map (`out_indices = (1,-2)`).
- **Auxiliary 2D head.**The intermediate feature map (early stage) is fed into a lightweight 2D convolutional head to produce per-slice auxiliary logits, which encourage the encoder to capture vessel-relevant spatial

cues before 3D aggregation. The auxiliary head is supervised with slice-level targets to stabilize early-layer learning.

- **2D→3D fusion.** Each volume `(B, C=1, D, H, W)` is **rearranged into `B·D` 2D slices**, encoded independently, then **reassembled** to `(B, C_feat, D, H′, W′)` before the 3D head. If the backbone expects 3 channels, single-channel inputs are channel-repeated.
- **3D temporal head.** A lightweight head (Conv3d-BN-ReLU ×2, `base_channels`) aggregates signals across depth.
- **Outputs.** Two 1×1×1 conv heads:
  - **Heatmap head:** `num_classes = 13` vessel locations (3D centerness maps)
  - **Offset head:** 3-channel sub-voxel offsets `(dz, dy, dx)`

Supervision targets

- Place a **3D Gaussian** peak (stride-aware) at each annotated center on the class-specific heatmap.
- Store **sub-voxel residual offsets** at the nearest grid cell and mark them with an **offset mask**.
- Scale coordinates from the original series size to the current `(D, H, W)`; map to the output grid using `(stride_d=1, stride_h=stride_w=16)` so the output is `D × H/16 × W/16`.
- Default **Gaussian σ = 0.2**.

Segmentation masks

Vessel masks are produced in physical space by training a 3D DynUNet on volumes resampled to 0.7 mm isotropic spacing. The provided voxel-level annotations of 13 vessel classes are merged into a single binary vessel mask for training. The resulting 3D masks are then converted to slice-wise 2D masks to supervise the auxiliary head.

Losses

We train with four terms and sum them with weights:

- **CenterNet-style focal loss** (weight = 1.0) on heatmaps (α=2, β=4).
- **L1 offset loss** (weight = 1.0) (only at valid center cells).
- **BCE-with-logits classification loss** (weight = 1.0) from series-level logits .
- **Auxiliary Dice & BCE loss** (weight = 0.5) on 2D auxiliary outputs when vessel segmentation masks are available; otherwise, it is set to zero, encouraging slice-wise feature consistency.

From voxelwise maps to series-level logits (for AUC & ensembling)

Given heatmap logits `H ∈ ℝ^{B×13×D×H′×W′}`:

- **Per-class series logits:** spatial **max** over `(D, H′, W′)` for each of the 13 classes.
- **Aneurysm Present (global) logit:** the **max over the 13** class logits.

  These **14 logits** drive the BCE loss and are exported for meta-ensembling.

Data pipeline & augmentations

- **Input.** Precomputed `.npy` volumes; `(D,H,W)` expanded to `(1,D,H,W)` — here D = 64.
- **Label scaling.** Rescale coordinates to current volume size; apply stride mapping for target grids.
- **Augmentations.** A **shared 2D affine warp** per volume (same transform for all slices): random rotation, scale, translation; vertical flip available (off by default).
- **Normalization.** Albumentations normalization → tensor conversion; channel repeat to 3-ch if needed.

Optimization & schedules

- **Optimizer.** AdamW (LR=2e-4, weight decay=1e-5), **AMP** mixed precision, **grad clipping** (=1.0).
- **LR schedule.** Default **CosineAnnealingLR** (`T_max = epochs`, `eta_min = 1e-6`); `StepLR` / `ReduceLROnPlateau` also supported.
- **Checkpointing.** Best model selected by **mean AUC** on validation.

Training configuration

- **Epochs:** 16
- **Batch size:** 4 (train) / 2 (val)
- **accumulate:**2
- **Workers:** 16
- **Strides:** depth `1`, lateral `16`
- **Gaussian σ:** 0.2
- **Backbone:** EfficientNetV2-S, `pretrained=True`, not frozen by default

Cross-validation & metrics

- K-fold protocol driven by the training CSV split.
- Compute **per-label ROC-AUC**, plus **mean** and **presence-weighted** AUC each epoch.
- Scheduler stepping and best-by-mean-AUC saving per fold; final summary reports per-fold and average AUC.

Flayer Results(without aux head)

Fold	loc_macro_auc	cls_auc	combined_mean
fold0	0.7762	0.7747	0.7754
fold1	0.7676	0.8069	0.7873
fold2	0.7508	0.7892	0.7700
fold3	0.7553	0.7778	0.7666
fold4	0.7734	0.7758	0.7746

Fold	loc_macro_auc	cls_auc	combined_mean
Average	**0.7647**	**0.7848**	**0.7748**

Flayer Results(with aux head)

Fold	loc_macro_auc	cls_auc	combined_mean
fold0	0.7779	0.7991	0.7885
fold1	0.7828	0.7987	0.7908
fold2	0.7601	0.8051	0.7826
fold3	0.7683	0.7826	0.7755
fold4	0.7586	0.7954	0.7770
Average	**0.7695**	**0.7962**	**0.7829**

Summary

Flayer converts 2D EfficientNetV2-S slice features into 3D evidence maps with a light 3D head, supervises them using CenterNet-style heatmaps + offsets, and produces robust **14-logit** series-level predictions (including an aggregated aneurysm present logit) that plug cleanly into our stacked meta-classifier.

Meta Classifier

We designed a stacked ensemble architecture to integrate predictions from **YOLO11m**, **YOLO11-EffV2s**, and **EffV2s-3D-CenterNet** models. Specifically, each meta-classifier receives the concatenated predictions from all base models as input and produces final predictions for both **aneurysm presence** and **location-specific detection**. This setup enables the model to assess the presence of an aneurysm at a given location by considering predictions from all locations across all models, effectively capturing the ensemble's collective reasoning about aneurysm presence.

```
metadata = np.array([age, sex])
X = np.concatenate([np.array([yolo11m_cls_preds[fold_id]]), yolo11m_loc_preds[fold_id],
                                np.array([effv2s_cls_preds[fold_id]]), effv2s_loc_preds[fold_id],
                                flayer_fold_preds[fold_id], metadata], axis=)[, :]
lgb_pred = predict_prob_lgb(X, fold_id)
xgb_pred = predict_prob_xgb(X, fold_id)
cat_pred = predict_prob_cat(X, fold_id)
```

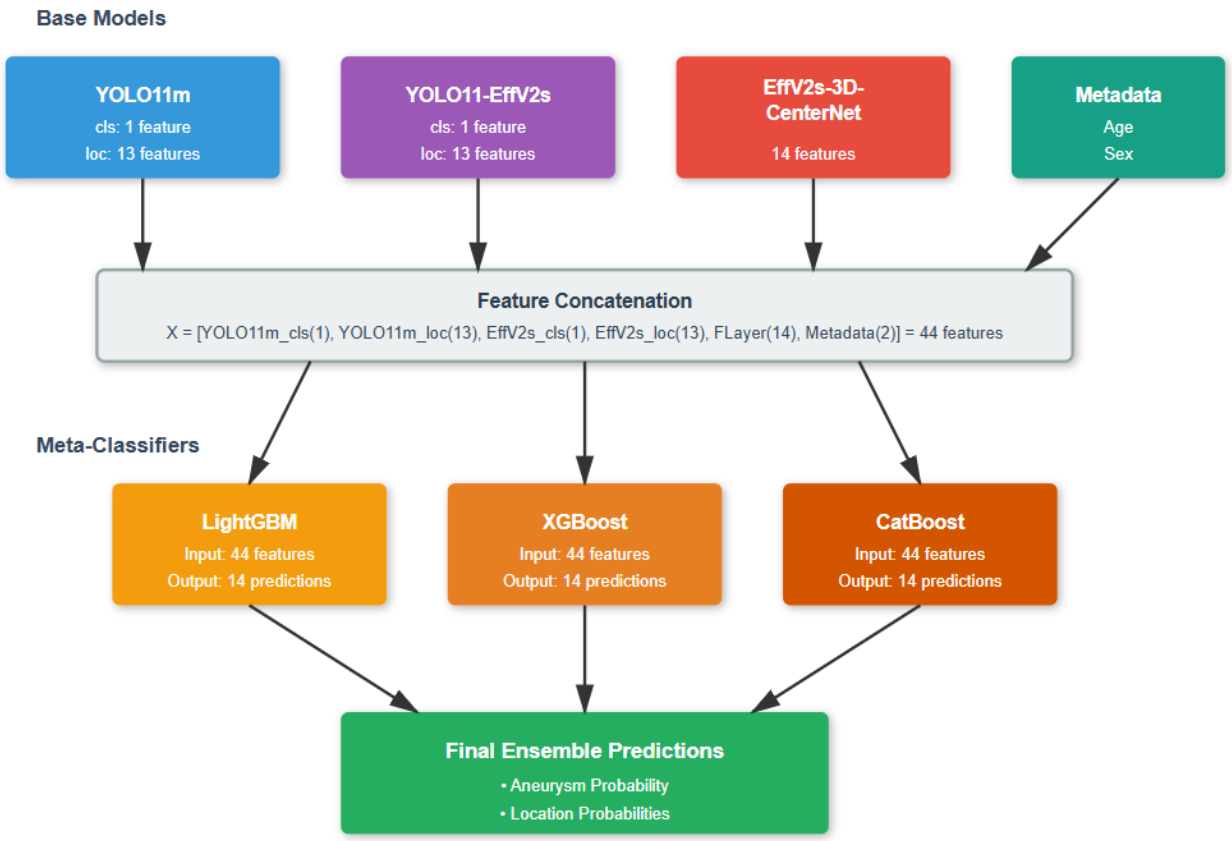
We experimented with two ensemble structures:

- **Series:** A single YOLO prediction combined with one Flayer prediction is fed into the ensemble model to produce the final output.

```
( features, [() + flayer() + meta()])
```

- **Parallel:** Two predictions from two YOLOs and one predictions from Flayer are processed in parallel and jointly used by the ensemble model to generate the final prediction.

```
( features, [yolo11m() + ()+ () + ()])
```



This comparison allows us to evaluate whether combining multiple feature streams in parallel provides additional complementary information beyond the sequential (series) configuration.

We found that combining predictions from different YOLO backbones and flayer in parallel provides better improvement than series in the ensemble CV score. This suggests that the GBDT effectively leverages each model's perception of aneurysm presence across locations to make accurate judgments. We were unable to complete the 5-fold submission due to issues encountered on the final day. Therefore, we present here our final selected result, which is based on the average of 2 folds.

	Series	Parallel
5 folds CV (average)	0.84	**0.852**
5 folds CV (nelder-mead optimized)	0.841	**0.858**
public score (average; 2 folds)	0.83	0.83082
private score (average; 2 folds)	0.81	**0.8230**

We've also tried Keypoint-Based Dual-Graph Predictor in Tom's BYU approach earlier and found it performed well in terms of ensemble CV performance. However, the YOLO-based feature extraction and preprocessing were too time-consuming and often led to timeouts. Therefore, we did not include this method in our final solution.

Codebase & Submission

Submission:

- **2x yolo + flayer + meta classifier (final ver)**
- kaggle models submission

Meta classifier training: **2x yolo + flayer meta training (final ver)**

Github: 9th-place-solution-yolo-RSNA-IAD

Supplemental Q&A

> **Ujjwal Pandey · 2025-10-16T05:24:13.120000**
>
> Hi, thanks for wonderful writeup. I have below questions.
>
> 1. Did you guys considred trying out reorientation or resampling of the niftii volume in pre-processing pipeline?
> 2. Any specific reason for usage of GBDT in the pipeline. If I am correct Age + Sex metadata is not there for test set. (Was it just matter of convinence?)
> 3. Any motivation for 2D or 2.5D based solution? (Was it because of mixture of thick and thin slices).
> 4. Did you guys had any attempt with pure E2E 3D based solution?

> **atom1231 · 2025-10-16T06:06:00.620000**
>
> > Did you guys had any attempt with pure E2E 3D based solution?
>
> The biggest issues encountered with a pure 3D solution are insufficient computational power (compute) and excessively long training times. The CenterNet within the solution is a compromised/truncated 3D solution designed to mitigate the constraints of compute and time.

> **Tom · 2025-10-16T08:08:12.570000**
>
> ```
>  specific reason    GBDT  the pipeline.  I am correct Age + Sex metadata   there  test . (Was it
> just matter  convinence?)
> ```
>
> The reason of using meta-classifiers is that we noticed when predicting a class, aggregating predictions from other classes can give a significant CV boost. Therefore, we believe the predicted probabilities of other classes can also serve as useful clues for identifying aneurysms in a specific locations.
>
> We noticed that the sample test data contain Age and Sex, but after submitting, with or without meta data the didn't change Leaderboard (LB) score much, while the cross-validation (CV) score showed a noticeable

difference.
Therefore, we decided to keep Age and Sex features, assuming that some data in hidden test set also includes them.

```
Any motivation for D .D solution? (Was it of mixture of thick thin slices).
```

In the first month, we found that the 3D classifier approach tended to overfit easily, as representing each voxel along the channel dimension made the model prone to overfitting. However, depth context is crucial for detecting irregular structural changes that could indicate an aneurysm, and slice-wise predictions not having expected result here. We found that using 3 local adjancent depths as the channel dimension to represent each pixel outperforms other previous methods. This approach proved effective for YOLO in capturing anomaly locations based on slight depth structure changes and achieved better CV compared to YOLO models that takes processed single slices repeated along RGB channels.